

Practical Experiment 4

Question 1:

Code:

```
#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

int main()
{
    char command[50];

    printf("Enter command: ");
    scanf("%s", command);

    int pid = fork();

    if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());
        execlp(command, command, NULL);
    }
```

```

else

{

    printf("Parent PID: %d\n", getpid());

    printf("Child PID: %d\n", pid);

    wait(NULL);

    printf("Child completed\n");

}

return 0;

}

```

Output :

```

venkat038@AYYAPPA:~$ nano spl.c
venkat038@AYYAPPA:~$ nano spl.c
venkat038@AYYAPPA:~$ gcc spl.c -o spl
venkat038@AYYAPPA:~$ ./spl
Enter command: ls
Parent PID: 627
Child PID: 628
Child PID: 628
1.c 6 8.c allow.c close.c exp2.c fork.c main.c read spl.c.save zombie_fixed
2.c 6.c 9 call command_executor exp3.c hello operating read.c write zombie_fixed.c
3.c 7 9.c call.c command_executor.c exp3.c hello.c output.txt snap write.c
4.c 7.c OSSP child display filename input.txt parent spl zombie
5.c 8 allow close exp2 fork main program5 spl.c zombie.c
Child completed

```

Question 2:

Code:

```

#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

int main()

```

```
{  
    char command[50];  
  
    printf("Enter command: ");  
    scanf("%s", command);  
  
    int pid = fork();  
  
    if (pid == 0)  
    {  
        printf("Child PID: %d\n", getpid());  
        execlp(command, command, NULL);  
    }  
    else  
    {  
        printf("Parent PID: %d\n", getpid());  
        printf("Child PID: %d\n", pid);  
  
        wait(NULL);  
  
        printf("Child completed\n");  
    }  
  
    return 0;  
}
```

Output:

```
Child completed
venkat038@AYYAPPA:~$ nano sp2.c
venkat038@AYYAPPA:~$ gcc sp2.c -o sp2
venkat038@AYYAPPA:~$ ./sp2
Enter command: date
Parent PID: 818
Child PID: 863
Child PID: 863
Mon Aug 24 12:56:50 UTC 2026
Child completed
venkat038@AYYAPPA:~$ |
```

Question 3:

Code:

```
#include <stdio.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
    int source, destination;
```

```
    char buffer[100];
```

```
    int n;
```

```
    source = open("source.txt", O_RDONLY);
```

```
    destination = open("copy.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```
n = read(source, buffer, sizeof(buffer));

write(destination, buffer, n);

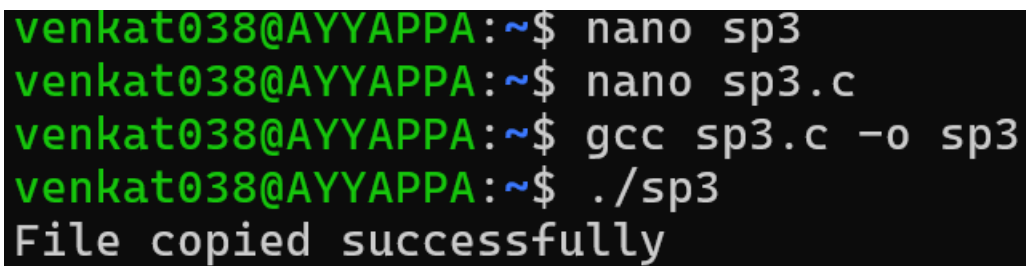
close(source);

close(destination);

printf("File copied successfully\n");

return 0;
}
```

Output:

A terminal window with a black background and green text. It shows a series of commands and their outputs. The prompt is 'venkat038@Ayyappa:~\$'. The commands are 'nano sp3', 'nano sp3.c', 'gcc sp3.c -o sp3', and './sp3'. The output of the last command is 'File copied successfully'.

```
venkat038@Ayyappa:~$ nano sp3
venkat038@Ayyappa:~$ nano sp3.c
venkat038@Ayyappa:~$ gcc sp3.c -o sp3
venkat038@Ayyappa:~$ ./sp3
File copied successfully
```

Question 4:

Code:

```
#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

int main()
```

```
{  
    int pid = fork();  
  
    if (pid == 0)  
    {  
        printf("Child Process\n");  
        printf("PID: %d\n", getpid());  
        printf("PPID: %d\n", getppid());  
  
        sleep(5);  
  
        printf("Child completed\n");  
    }  
    else  
    {  
        printf("Parent Process\n");  
        printf("PID: %d\n", getpid());  
        printf("Child PID: %d\n", pid);  
  
        wait(NULL);  
  
        printf("Parent completed\n");  
    }  
  
    return 0;
```

}

Output:

```
venkat038@AYYAPPA:~$ nano sp4.c
venkat038@AYYAPPA:~$ gcc sp4.c -o sp4
venkat038@AYYAPPA:~$ ./sp4
Parent Process
PID: 1072
Child PID: 1073
Child Process
PID: 1073
PPID: 1072
Child completed
Parent completed
venkat038@AYYAPPA:~$ |
```